Unlocking algorithm

The lesson unlocking algorithm in determines whether a student can access the next lesson based on their learning progress. It uses conditional statements and sequential progression rules to check whether the student has achieved the required number of correct answers in the previous lesson. The function getRequiredCorrect(lessonId) determines the required number of correct answers for a particular lesson. The system also checks whether the student has passed the pretest, if the lesson requires one. If the student meets the practice requirement and any required pretest condition, the next lesson can be unlocked. If the requirements are not met, the lesson remains locked. If the system cannot load the student's progress, later lessons remain locked to prevent access without verified progress. This algorithm helps organize lessons in a structured sequence and encourages students to complete the required activities before advancing.

CODE:

```
const isLessonUnlocked = (
  index: number,
  lessons: { lessonId: string }[],
): boolean => {
  return lessons
    .slice(0, index)
    .every((lesson) =>
      isLessonComplete(lesson.lessonId)
    );
};
```

Where `isLessonComplete` is:

```
const isLessonComplete = (lessonId: string): boolean => {
  const practice = practiceProgress[lessonId];
  const required = getRequiredCorrect(lessonId);

  const practicePassed =
    practice &&
    required !== null &&
    practice.correctAnswers >= required;

  const pretestPassed =
    !hasPretest(lessonId) ||
    pretestProgress[lessonId]?.passed === true;

  return Boolean(practicePassed && pretestPassed);
};
```

In one compact version:

```
const isLessonUnlocked = (index: number) =>
  lessons
    .slice(0, index)
    .every(({ lessonId }) =>
      isPracticeComplete(lessonId) &&
      isPretestPassed(lessonId)
```

);

FER

The facial expression recognition algorithm estimates the student's facial expression by combining face detection, facial-feature measurements, and predefined classification rules. It is implemented in PracticeCamera.tsx and uses the face detector to obtain information about facial features, such as eye openness, mouth opening, eyebrow position, and smile probability. The application applies specific conditions to classify expressions such as happy, neutral, shocked, angry, sad, and confused. For example, a smile probability of at least 0.70 is used to classify a face as happy, while a probability of 0.35 or lower can indicate a neutral expression, provided an earlier expression rule has not already matched. Other expressions are identified through combinations of facial measurements, such as wide-open eyes and an open mouth for a shocked expression. To reduce inconsistent results from individual camera frames, the application checks the latest three expression samples and confirms the expected expression when at least two samples match. This method is a rule-based facial expression classification approach rather than a separately trained deep-learning emotion classifier. It estimates visible facial patterns and does not establish a person's actual emotional state.

```
function classifyExpression(face: Face): PracticeExpression {
  const smilingProbability = face.smilingProbability;

  if (smilingProbability != null && smilingProbability >= 0.7) {
    return "happy";
```

```
  }

  if (isShockedFace(face)) return "shocked";
  if (isAngryFace(face)) return "angry";
  if (isSadFace(face)) return "sad";
  if (isConfusedFace(face)) return "confused";

  if (smilingProbability != null && smilingProbability <= 0.35) {
    return "neutral";
  }

  return "unknown";
}
```

Voice Recognition

The voice recognition algorithm in checks whether the words spoken by the student match the expected words in the lesson. It uses the expo-speech-recognition library to convert speech into text, with the Filipino. Once the speech recognition engine produces a transcript, the application processes the text using a normalization function called normalizeWord(). This function converts letters to lowercase, removes accent marks and punctuation, and standardizes the text to make comparison more consistent. The application then compares the recognized words with the expected words in their corresponding positions. If all expected words match, the voice recognition stage passes; if one or more words differ, the stage fails. This approach combines automatic speech recognition with text normalization and exact word matching. However, it

relies on the transcript produced by the speech recognition engine, so it does not independently measure every aspect of pronunciation, such as articulation, accent, or linguistic stress.

CODE:

```
const normalizeWord = (value: string) =>
  value
    .toLowerCase()
    .normalize("NFD")
    .replace(/[\u0300-\u036f]/g, "")
    .replace(/[^a-z0-9]/g, "")
    .trim();
```

Pitch Recognition

The pitch recognition algorithm analyzes the student's voice to estimate its fundamental frequency (F0), which is measured in hertz (Hz). The application uses the YIN algorithm through the Pitchfinder library, configured with a sample rate of 16,000 Hz. The recorded audio is divided into smaller frames, and YIN analyzes the signal to estimate its repeating period and calculate the corresponding fundamental frequency. The application then filters unusual measurements using a median-based approach and considers valid frequency estimates within the range of 60 to 450 Hz. It classifies frequencies from 60 to 140 Hz as low pitch, frequencies above 140 Hz up to 220 Hz as medium pitch, and frequencies above 220 Hz up to 450 Hz as high pitch. If there are not enough valid and stable measurements, the system can return an unknown classification rather than accepting an unreliable result. YIN provides the fundamental frequency estimation, while the frequency thresholds and validation conditions are additional rules implemented. Although this method can classify pitch levels, those categories alone cannot

establish whether a student has correctly pronounced the linguistic stress of a Tagalog, Kapampangan, or Waray word. Verifying word stress would require appropriate language-specific stress criteria or reference pronunciations.

```
const SAMPLE_RATE = 16000;


const detectPitch = Pitchfinder.YIN({
  sampleRate: SAMPLE_RATE,
});
```